

Maestro

Contents

- [Maestro](#)
- [Purpose](#)
- [Instructions](#)
- [Cautions](#)
- [Message Handlers](#)
- [Requirements](#)
- [Operation](#)
- [Variations](#)
- [Future](#)

Purpose

maestro's purpose is to get results of simulations faster, and with less human work, than would be possible using **mc run** alone. It does so by running simulations in parallel.

maestro is the conductor of all the individual simulations; its modules (Python code) are in the **symphony** directory.

Most results are available directly under the main project directory and full results are under the **parallel** directory. The parallelization is by **.inp** file; simulations for each **.inp** run separately and simultaneously.

maestro currently uses a command-line interface only.

The monte-carlo system and the **Fortran** model it runs are not designed for multiple parallel runs. Simultaneous runs in the same directory may step on each other, producing either obvious crashes or more subtle corruption. To avoid such difficulties, **maestro** makes multiple copies of the project directory; each run is in a separate directory.

maestro is a stopgap solution because the ideal solution, changing **mccli** and the programs it invokes, especially the **Fortran** model, to run safely in parallel in a single directory, seems riskier and more time-consuming.

Instructions

1. Install **mccli** as described in the general [README](#), if you haven't already. This code is in the **parallel** branch, which you can switch to with your favorite **git** tool or **git checkout parallel** from a terminal.
2. Review the [Requirements](#) below. You need a beefy system, and if you are on **MS-Windows** you need a somewhat recent version and/or administrative rights.

3. Create a project directory, separate from the code, and set up the project for analysis as usual. Try to avoid extraneous files, as they are likely to be copied many times. If you are on **MS-Windows**, success is more likely if the project is on a local drive.
4. If you are using a **Python** virtual environment (you are if you followed the standard **mccli** instructions), activate it.
5. Execute subsequent steps from a terminal in the top of the project directory. Running from the **mccli** directory will not work.
6. Run **mc init** as usual, selecting *all* scenarios (**.inp** files) you wish to consider. Then do any post-init setup necessary to get the project ready for a usual **mc run**.
7. Check the code for **maestro.py** in light of the **Cautions** below, and make any changes necessary.
8. Run **python maestro.py**. The **python** is there to ensure you get the version associated with your virtual environment.
9. When it finishes, if you don't want any of the extra information, delete the **parallel** directory and all its children. This will retain the **MC_XXX** directories under the main project with the **MC** results. Ordinarily, most of the space occupied will be there.

Cautions

This is alpha-quality code, currently in development. You should review the decisions in the code for appropriateness for your purposes and system:

1. Are the directory choices appropriate for your project?
2. Are the options used to invoke the monte-carlo simulation, in particular the number of simulations, the seed, and the use of **--overwrite** appropriate?
3. Does the active code either copy the files with **prepare()** on first invocation or use **prepare_basics()** if the working directories are already set?
4. Does it use the **message handlers/loggers** that you want?
5. Does it setup and invoke the runs?

You may think that many more of the files and directories copied could be symbolic links as well, thereby saving space and time. **mccli** and the software it calls *write* to most directories, even ones with "input" in the name. It writes to many "input" files. So if you get the urge to convert copies to links, resist it.

maestro.py depends upon **mc run** being invoked with **--json**; do not remove that option.

Message Handlers

maestro processes terminal output from the individual runs as **JSON** messages. It can use any combination of the following (from **symphony/message_handlers.py**):

- **StupidLogFile(aPath)** will dump all messages as text representation of **JSON** to the file at location **aPath**, which may be a string or a **pathlib.Path** object.

- `DumbTerminalLog()` prints the messages as text for `JSON` to the terminal. This can be used in a dumb terminal because it makes no attempt to colorize, ring bells, or address different locations on the screen.
- `TerminalTimerLog(niter, nrun, updateInterval=timedelta(minutes=5))` also requires only a dumb terminal, but is much more selective and intelligent in its output. It records timing information for each of `niter` iterations of the model across `nrun` parallel runs. Every `updateInterval` it prints a summary progress report, including estimated time of completion for the entire batch, as well as the individual runs. These estimates are based on the average time across all iterations, and so will differ from those of `mc run`, which uses the time of the most recent iteration only to estimate time remaining.

Requirements

Your system must have adequate RAM, disk space, and CPUs to handle the parallel runs. In testing, each run required 1.6GB of RAM.

This was developed for `MS-Windows` and might have issues on other platforms.

On the other hand, if you are on `MS-Windows` you may have problems because `maestro.py` creates symbolic links. That's no problem on `*nix` variants, including `MacOS`, but `MS-Windows` makes it difficult. It has been made less difficult over time, with `Windows 10` from around 2017 on, as well as `Windows 11`, being easier. Assuming you have at least that, here are the rules as best I can tell:

1. If you run ``maestro.py`` with administrative rights, there should be no problem.
2. If you enable developer mode, there should be no problem.
3. If your user has ``Create symbolic links`` rights, it will probably work, although some things I've read indicate this may need to be combined with 2.

Note that to set developer mode (#2) or add rights to a user (#3) requires administrative privileges, although once they have been done you can run `maestro.py` as a regular user.

With admin rights, `MS-Windows` has had symbolic links from `Vista` onward, though I think initially they were only available for the operating system kernel.

You are more likely to run into trouble if you attempt to run `maestro` for a project on a network drive.

Practically, try the following steps in order until one works:

1. Run `maestro.py` and see if it works. If it fails with an error that it can't create a symbolic link, try next option.
2. If you can, run `maestro.py` as an administrator. Typically files have a "run as administrator" option when you right click on them, if you have permissions. Use that to launch your favorite command prompt and try again. Do *not* run `maestro.py` directly from the file explorer, as this will likely not have the `Python` virtual environment set properly.
3. Have an administrator give you `Create symbolic links` rights.

4. Have an administrator enable "Developer Mode" on your machine. I put this last because it has more wide-ranging effects than 3, including giving everyone rights to create symbolic links.

See [this useful guide](#) for an illustrated walk-through of all 3 ways of enabling symlinks.

Bear in mind that

- Symbolic links or Developer Mode weaken system security.
- To get recently added rights you must log out and then log back in.
- Unlike **Unix** links, **MS-Windows** distinguishes links to directories from links to files.
- Even when using a link, it may matter whether the source (where the link is) or the target is local or remote, where "remote" includes network drives that you have mapped. You can check with following command:

```
PS C:\Users\rdboylan> fsutil behavior query SymlinkEvaluation
Local to local symbolic links are enabled.
Local to remote symbolic links are enabled.
Remote to local symbolic links are disabled.
Remote to remote symbolic links are disabled.
```

That indicates symbolic links on network shares would not work for me. The name suggests this is about whether I can use them; I might still be able to create them.

The developer [Notes](#) provide further information about symbolic links and what to do if none of the options above work.

Operation

maestro creates a **parallel** subdirectory of the main project. For each **run**, a single execution of **mc run**, it creates a directory under **parallel** and copies the project files to that subdirectory. There is one run for each scenario (**.inp** file), in a directory named after the name of the **.inp** file without the extension. Each is set to simulate only one of the **.inp** files, even though in **mc init** you specified a simulation that would ordinarily run all, serially, within the same simulation.

There will be **MC** directories under the subprojects; these are actually links to directories named **MC_xxx** under the main project. This means that when the runs finish one can simply look to those directories for the results.

So the original setup looks like this (simplified):

```
myproject
myproject/MC
```

Supposing there were 2 scenarios, a and b, after setup it would be

```
myproject
myproject/MC
myproject/MC_a
myproject/MC_b
myproject/parallel
myproject/parallel/a
myproject/parallel/a/MC (links to myproject/MC_a)
myproject/parallel/b
myproject/parallel/b/MC (links to myproject/MC_b)
```

with all relevant files and directories from `myproject/` copied to the `a` and `b` directories, modified as necessary.

`maestro` then launches `mc run` in each subproject `parallel/xxx/`, monitors the progress, and reports the results. The exact behavior depends on the `message handlers` (`symphony/message_handlers.py`) installed in the `Switchboard` of the main program.

With standard code (from `symphony.message_handlers.StupidLogFile`), `parallel/runlog.txt` will have all the messages generated by all the runs. If you are not also dumping those messages to the terminal (`symphony.message_handlers.DumbTerminalLog`), or getting progress reports on a terminal (`symphony.message_handlers.TerminalTimerLog`), you can monitor the file as the program runs to see progress. Because of caching, messages may not be visible instantly. All messages are in `JSON` format.

Variations

As noted earlier, if `prepare()` has already set up the copies, just call `prepare_basic()` to avoid recopying, which might or might not produce errors or toss results of previous runs.

`prepare(basics, stemcell="/some/path")` will make a single copy of the project directory to `/some/path`; it will not create the `parallel` directory or anything under it, and it will not modify the request to simulate multiple `.inp` files. This exists mostly as a convenience for testing; one can then do a regular run of `maestro` from the newly created copy.

Future

Perhaps one day there will be a GUI, or at least output that treats the terminal as a 2 dimensional surface, `curses` style (Python `curses` is not available on `MS-Windows` and an implementation for `Windows` seems unmaintained and not working).

The logs could go to a database instead of a file.

A richer range of statistics and estimates for the runs might be helpful.

Each run does all iterations for a single scenario/`.inp` file. `maestro` could allow several input files in a single run, for example to limit the number of parallel runs. Or, the runs could be further broken down so that each does

only a portion of the iterations to be covered.